LABORATORY RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 4
Student Name: N. Samhavya
Roll No.: A24126552099
Date: 29-08-2026

---

1. TITLE

JavaScript Basics – Arrays & Functions

---

2. AIM

To understand and implement the basic concepts of **JavaScript arrays, functions, array methods, loops, and classes** by creating simple programs and observing their output.

---

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of arrays in JavaScript.
- To create and initialize arrays with values.
- To access and modify array elements using index numbers.
- To determine the length of an array.
- To use array methods such as `push()`, `pop()`, `shift()`, and `unshift()`.
- To iterate through array elements using a `for...of` loop.
- To understand the creation and use of functions in JavaScript.
- To pass arrays and values as function arguments.
- To add and remove elements from an array using functions.
- To understand classes, constructors, methods, and private fields in JavaScript.

---

4. THEORY

JavaScript is a high-level programming language commonly used to add dynamic behavior and interactivity to web applications. It supports various programming concepts such as variables, arrays, functions, objects, classes, loops, and methods.

Arrays

An **array** is a collection of multiple values stored in a single variable. Array elements are accessed using an index, and the index starts from **0**.

For example:

```
const colors = ["Red", "Green", "Blue"];
```

Here, `Red` is at index `0`, `Green` is at index `1`, and `Blue` is at index `2`.

JavaScript provides several built-in array methods:

- `push()` – adds an element to the end of an array.
- `pop()` – removes the last element.
- `shift()` – removes the first element.
- `unshift()` – adds an element to the beginning.
- `length` – returns the number of elements in an array.

Arrays can also be processed using loops such as the `for...of` loop.

Functions

A **function** is a reusable block of code designed to perform a particular task. Functions can accept parameters and can be called multiple times with different values.

In this experiment, functions are used to:

- Display shopping cart items.
- Add a new item to the cart.
- Remove the last item from the cart.

Classes and Private Fields

JavaScript also supports **classes**, which provide a way to create objects with properties and methods. A constructor is used to initialize an object.

The `#` prefix is used to declare a **private class field**. A private field cannot be accessed directly from outside the class. In the BankAccount program, `#balance` is private and is accessed through the `getBalance()` method.

The basic concepts demonstrated in this experiment are:

Arrays → Array Methods → Loops → Functions → Function Calls → Classes → Private Fields

---

5. ALGORITHM / PROCEDURE

1. Create a JavaScript project or folder.

2. Create the required JavaScript files.
3. Create an array containing initial values.
4. Display the original array using `console.log()`.
5. Access individual array elements using index numbers.
6. Find the total number of elements using the `length` property.
7. Modify an element using its index.
8. Add an element using `push()`.
9. Remove the last element using `pop()`.
10. Remove the first element using `shift()`.
11. Add an element to the beginning using `unshift()`.
12. Iterate through the array using a `for...of` loop.
13. Create reusable functions for shopping cart operations.
14. Pass the shopping cart array as a function argument.
15. Add and remove items using separate functions.
16. Create a `BankAccount` class with a private balance field.
17. Create a constructor to initialize the account owner.
18. Create methods to deposit money and retrieve the balance.
19. Execute the programs using a JavaScript runtime/browser console.
20. Observe and record the output of all three programs.

---

6. PROGRAM

Program 1: array.js

```
// 1. Creating an array with initial values
const colors = ["Red", "Green", "Blue"];
console.log("Original List:", colors);

// 2. Accessing items using index numbers (Starts at 0)
console.log("First color (index 0):", colors[0]); // Output: Red
console.log("Second color (index 1):", colors[1]); // Output: Green

// 3. Finding the total number of items
console.log("Total colors in list:", colors.length); // Output: 3

// 4. Updating an item at a specific index
colors[1] = "Yellow"; // Changes "Green" to "Yellow"
console.log("After updating index 1:", colors);

// 5. Adding a new item to the END of the array
colors.push("Purple");
console.log("After adding to the end (.push):", colors);

// 6. Removing the last item from the array
const removedColor = colors.pop(); // Removes "Purple" and saves it
```

```
console.log("Removed color:", removedColor);
console.log("Array after removing last item (.pop):", colors);

const remove = colors.shift(); // Removes 1st element from array
console.log("Remove:",remove)

colors.unshift("Pink") //Adds element at 1st
console.log("adding at start",colors)

// 7. Looping through the array to see every item
console.log("\n--- Printing all colors one by one ---");
for (const color of colors) {
  console.log(`Color: ${color}`);
}
```

Program 2: oop.js

```
class BankAccount {
 // 1. Declare the private field using the '#' prefix
 #balance = 0;

 constructor(owner) {
  this.owner = owner; // Public property
 }

 // 2. Control access via a public method (Getter)
 getBalance() {
  return this.#balance;
 }

 // 3. Validate modifications via a public method (Setter-like operation)
 deposit(amount) {
  if (amount > 0) {
   this.#balance += amount;
   console.log(`Deposited ${amount}.`);
  } else {
   console.log("Invalid deposit amount.");
  }
 }
}

// --- Usage ---
const myAccount = new BankAccount("Alex");

myAccount.deposit(500);      // Output: Deposited 500.
console.log(myAccount.getBalance()); // Output: 500
```

Program 3: script.js

```
const shoppingCart = ["Laptop", "Headphones", "Mouse Pad"];
// Function to print all items currently in the cart
function viewCart(cartArray) {
  console.log(`\n Your Cart (${cartArray.length} items)`);
  if (cartArray.length === 0) {
    console.log("Your cart is empty!");
    return; // Stops the function early if cart is empty
  }
  for (let i = 0; i < cartArray.length; i++) {
    console.log(`${i + 1}. ${cartArray[i]}`);
  }
}
// Function to add a new item to the cart
function addItem(cartArray, item) {
  cartArray.push(item);
  console.log(`Added to cart: ${item}`);
}
// Function to remove the very last item added
function removeLastItem(cartArray) {
  if (cartArray.length > 0) {
    const removed = cartArray.pop();
    console.log(`Removed from cart: ${removed}`);
  } else {
    console.log("Nothing to remove. Cart is already empty!");
  }
}
// 3. RUNNING THE PROGRAM (Calling the functions)
viewCart(shoppingCart);      // Shows the initial 3 items
addItem(shoppingCart, "Keyboard"); // Adds a 4th item
viewCart(shoppingCart);      // Shows updated list
removeLastItem(shoppingCart); // Removes "Keyboard"
viewCart(shoppingCart);      // Shows final list
```

---

7. CODE EXPLANATION

Code / Concept	Explanation
`const colors = [...]`	Creates an array containing initial color values.
`colors[0]`	Accesses the first element of the array.
`colors.length`	Returns the total number of elements in the array.
`colors[1] = "Yellow"`	Updates the element at index 1.
`push()`	Adds a new element to the end of an array.
`pop()`	Removes and returns the last element of an array.

Code / Concept	Explanation
`shift()`	Removes and returns the first element of an array.
`unshift()`	Adds a new element to the beginning of an array.
`for...of`	Iterates through each element of an array.
`console.log()`	Displays output in the console.
`class BankAccount`	Defines a class named `BankAccount`.
`#balance`	Declares a private class field.
`constructor()`	Initializes an object when it is created.
`this.owner`	Stores the account owner's name as a public property.
`getBalance()`	Returns the value of the private balance field.
`deposit()`	Adds a valid amount to the account balance.
`if`	Performs conditional checking.
`new BankAccount()`	Creates a new object from the `BankAccount` class.
`function viewCart()`	Defines a function to display shopping cart items.
`function addItem()`	Defines a function to add an item to the cart.
`function removeLastItem()`	Defines a function to remove the last cart item.
`return`	Exits the function when the cart is empty.
Function parameter	Allows data such as the cart array or item to be passed into a function.
`cartArray.push(item)`	Adds an item to the shopping cart.
`cartArray.pop()`	Removes the last item from the shopping cart.

---

8. TEST CASES AND EXECUTION DATA

The three JavaScript programs were executed and their console outputs were verified.

Program 1: Array Operations

Test Case	Operation	Expected Result	Actual Result	Status
TC-01	Create colors array	Array should contain Red, Green, Blue	Array created successfully	**Pass**
TC-02	Access index 0	Red should be displayed	Red displayed	**Pass**
TC-03	Access index 1	Green should be displayed	Green displayed	**Pass**
TC-04	Find array length	Length should be 3	Length displayed as 3	**Pass**
TC-05	Update index 1	Green should change to Yellow	Yellow displayed	**Pass**
TC-06	Use `push()`	Purple should be added at the end	Purple added successfully	**Pass**
TC-07	Use `pop()`	Purple should be removed	Purple removed successfully	**Pass**

TC-08	Use `shift()`	First element should be removed	Red removed successfully	**Pass**
TC-09	Use `unshift()`	Pink should be added at the beginning	Pink added successfully	**Pass**
TC-10	Use `for...of` loop	All remaining colors should be displayed	Colors displayed individually	**Pass**

Program 2: BankAccount Class

Test Case	Operation / Input	Expected Result	Actual Result	Status
TC-11	Create BankAccount for Alex	Account object should be created	Account created successfully	**Pass**
TC-12	Deposit `500`	Balance should increase by 500	`Deposited 500.` displayed	**Pass**
TC-13	Call `getBalance()`	Balance should be 500	`500` displayed	**Pass**

Program 3: Shopping Cart Functions

Test Case	Operation / Input	Expected Result	Actual Result	Status
TC-14	Display initial cart	Three items should be displayed	Three items displayed	**Pass**
TC-15	Add `Keyboard`	Keyboard should be added to cart	Keyboard added successfully	**Pass**
TC-16	Display updated cart	Four items should be displayed	Four items displayed	**Pass**
TC-17	Remove last item	Keyboard should be removed	Keyboard removed successfully	**Pass**
TC-18	Display final cart	Original three items should remain	Three items displayed	**Pass**

9. OUTPUT

The three JavaScript programs were executed successfully. The corresponding console outputs are shown below.

Output 1: Array Operations – array.js

```
PS C:\Users\anits-csm\Desktop\csmb99> node array.js
Original List: [ 'Red', 'Green', 'Blue' ]
First color (index 0): Red
Second color (index 1): Green
Total colors in list: 3
After updating index 1: [ 'Red', 'Yellow', 'Blue' ]
After adding to the end (.push): [ 'Red', 'Yellow', 'Blue', 'Purple' ]
Removed color: Purple
Array after removing last item (.pop): [ 'Red', 'Yellow', 'Blue' ]
Remove: Red
adding at start [ 'Pink', 'Yellow', 'Blue' ]

--- Printing all colors one by one ---
Color: Pink
Color: Yellow
Color: Blue
PS C:\Users\anits-csm\Desktop\csmb99>
```

Figure 1: Output of Array Operations using JavaScript

The output demonstrates array creation, indexing, length calculation, updating, adding, removing, and looping operations.

Output 2: Bank Account Class – oop.js

```
Problems    Output    Debug Console    Terminal    Ports

PS C:\Users\anits-csm\Desktop\csmb99> node oop.js
Deposited 500.
500
PS C:\Users\anits-csm\Desktop\csmb99>
```

Figure 2: Output of BankAccount Class using JavaScript

The output shows successful deposit of `500` into the bank account and retrieval of the account balance using a public method.

Output 3: Shopping Cart Functions – script.js

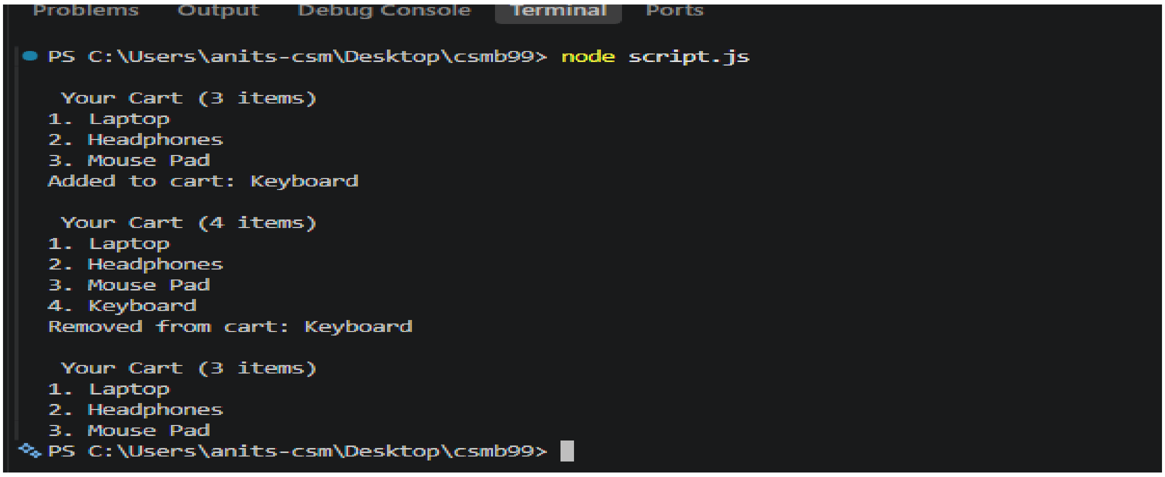

Figure 3: Output of Shopping Cart Operations using JavaScript Functions

The output demonstrates displaying the initial cart, adding an item, displaying the updated cart, removing the last item, and displaying the final cart.

---

10. RESULT

Thus, the basic concepts of **JavaScript arrays and functions** were successfully implemented and executed. Array operations such as indexing, updating, push(), pop(), shift(), unshift(), and iteration were demonstrated. Functions were used to perform shopping cart operations, and a JavaScript class with a private field was implemented using the `#` syntax. All three programs were executed successfully and the expected outputs were obtained.